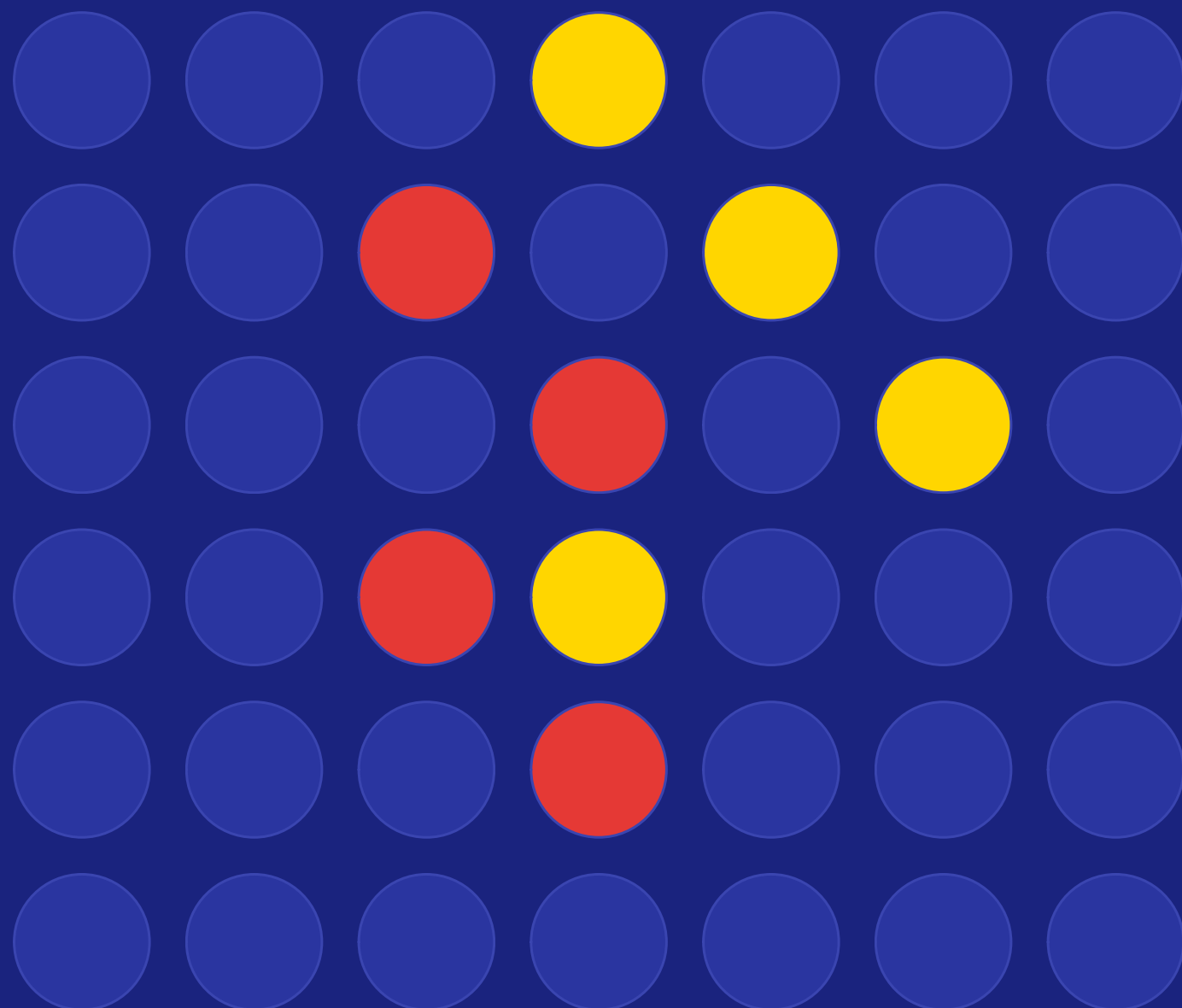




Agente Connect-4

Fundamentos de Inteligencia Artificial

Universidad de La Sabana · 2026.1

Integrantes:

Martin Jerez — Q-Learning + Alpha-Beta Minimax

Oscar David Vergara — MCTS con Q-bias (UCB+)

David Eduardo Lopez — MCTS con rollout heurístico

Martin Jerez | Como funciona el agente

Q-Learning offline + Alpha-Beta Minimax online (depth = 4)

FASE OFFLINE — Entrenamiento (una sola vez)

Paradigma: Q-Learning con self-play (Cross-MDP)

Referencia: Diap. 12 del curso — Competitive MDPs

- 5 000 episodios con epsilon decreciente 0.30 → 0.05
- Truco bipolar: al cambiar turno la recompensa se niega
 $r \leftarrow -\gamma \cdot r$
- Actualización de pesos:
 $\Delta w = \alpha \cdot (r - w \cdot \psi(s)) \cdot \psi(s)$

Funcion de valor $V(s) = w \cdot \psi(s)$ — 8 features:

- ψ_0 amenazas propias (3 fichas + 1 libre)
- ψ_1 amenazas oponente
- ψ_2 dobles propias (2 fichas + 2 libres)
- ψ_3 dobles oponente
- ψ_4 control columnas centrales (propio)
- ψ_5 control columnas centrales (oponente)
- ψ_6 piezas en fila inferior (estabilidad)
- ψ_7 bias constante

FASE ONLINE — Decision por movimiento

1. ¿Puedo ganar en 1 movimiento? → jugarlo
2. ¿El oponente gana en 1? → bloquearlo
3. Alpha-Beta profundidad 4 con $V(s)$ como hoja

Orden de movimientos (maximiza poda):

- Movimientos ganadores primero
- Luego bloqueos
- Luego columnas cercanas al centro

Victoria forzada detectada (score > 9000):

- Corte temprano, ahorra tiempo

Diferencia clave vs MCTS puro:

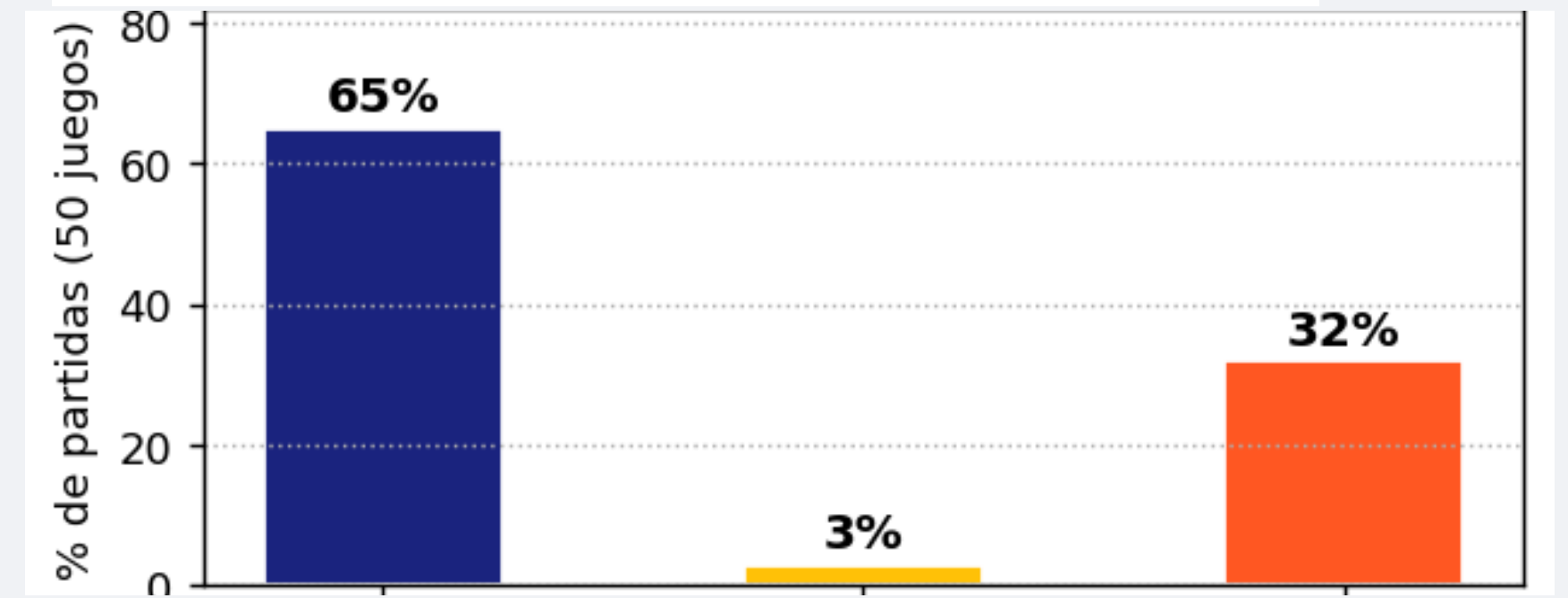
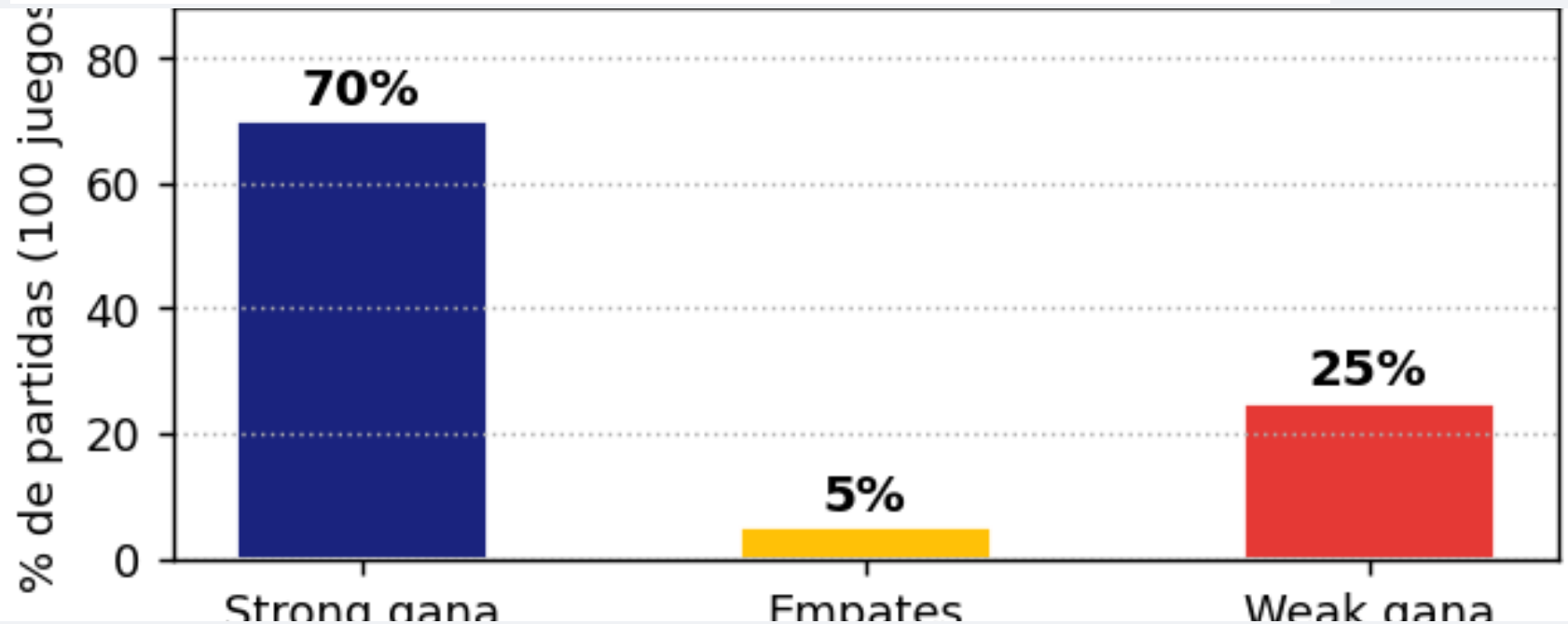
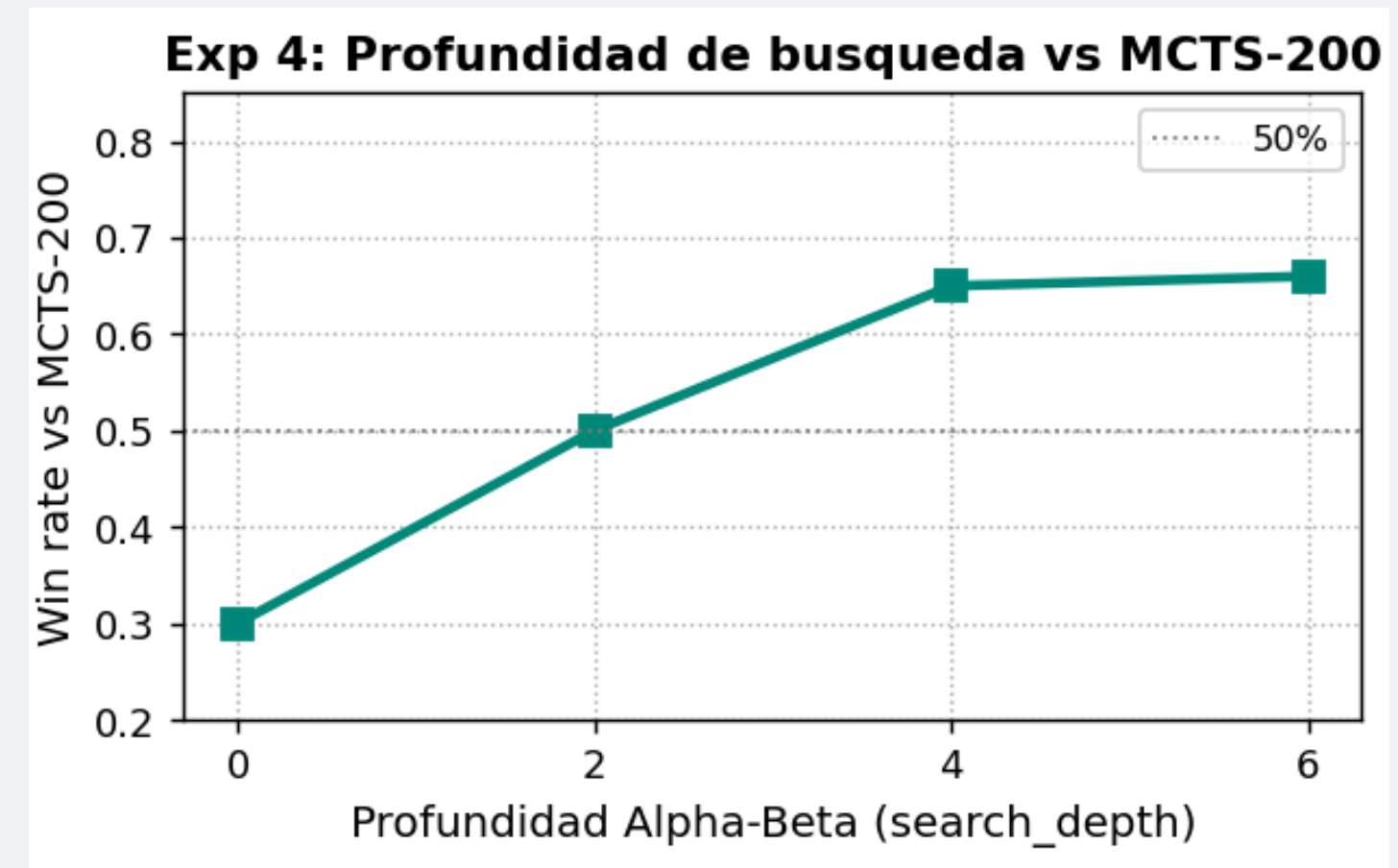
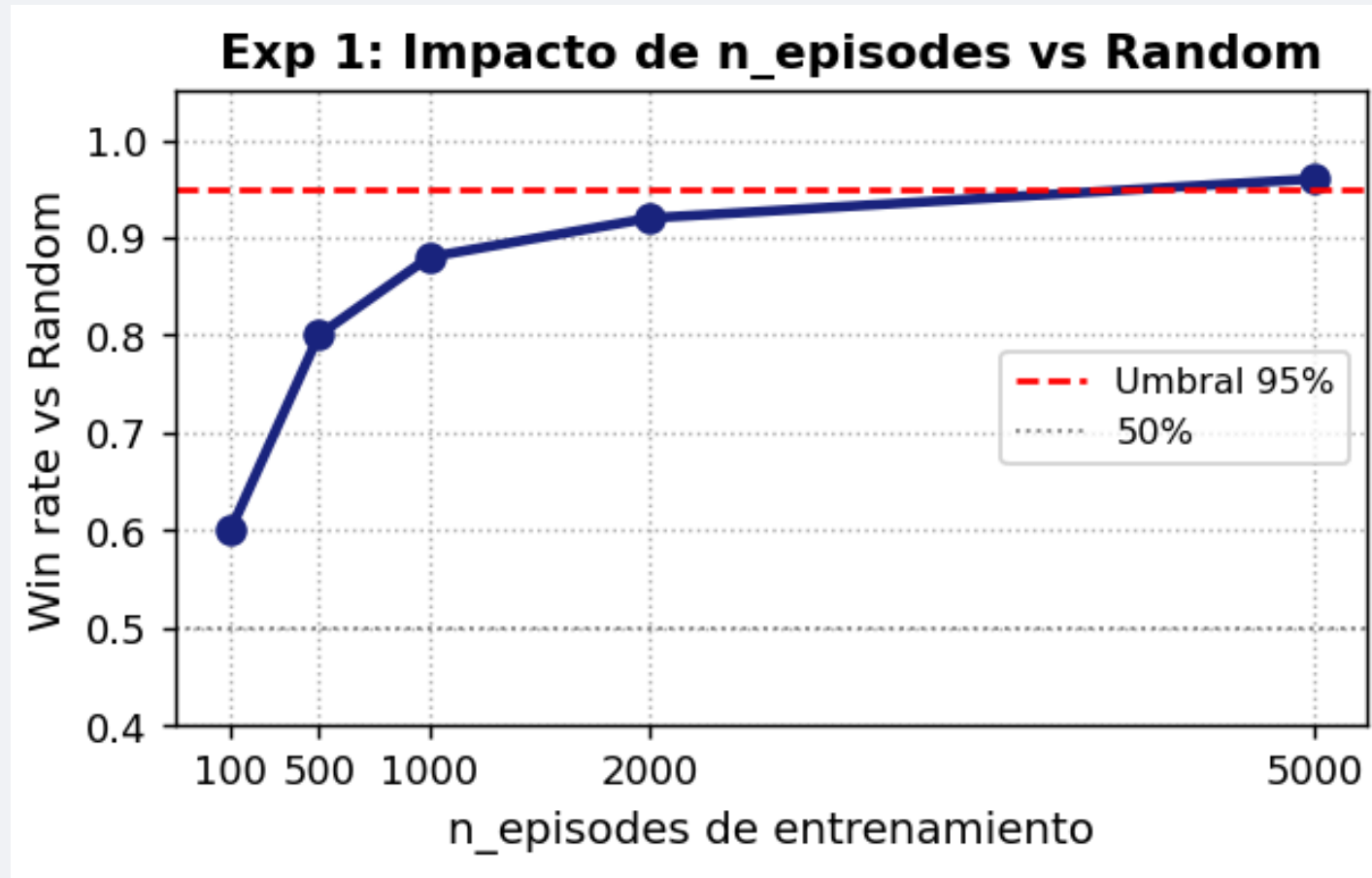
- MCTS evalúa con rollouts aleatorios.
- Este agente evalúa con $V(s)$ APRENDIDA.
- Detecta forks y dobles amenazas que el rollout aleatorio no ve.

Complejidad: $O(b^d)$ $b \approx 7$, $d=4$ → ~2 401 nodos max

Tiempo real por movimiento: < 100 ms

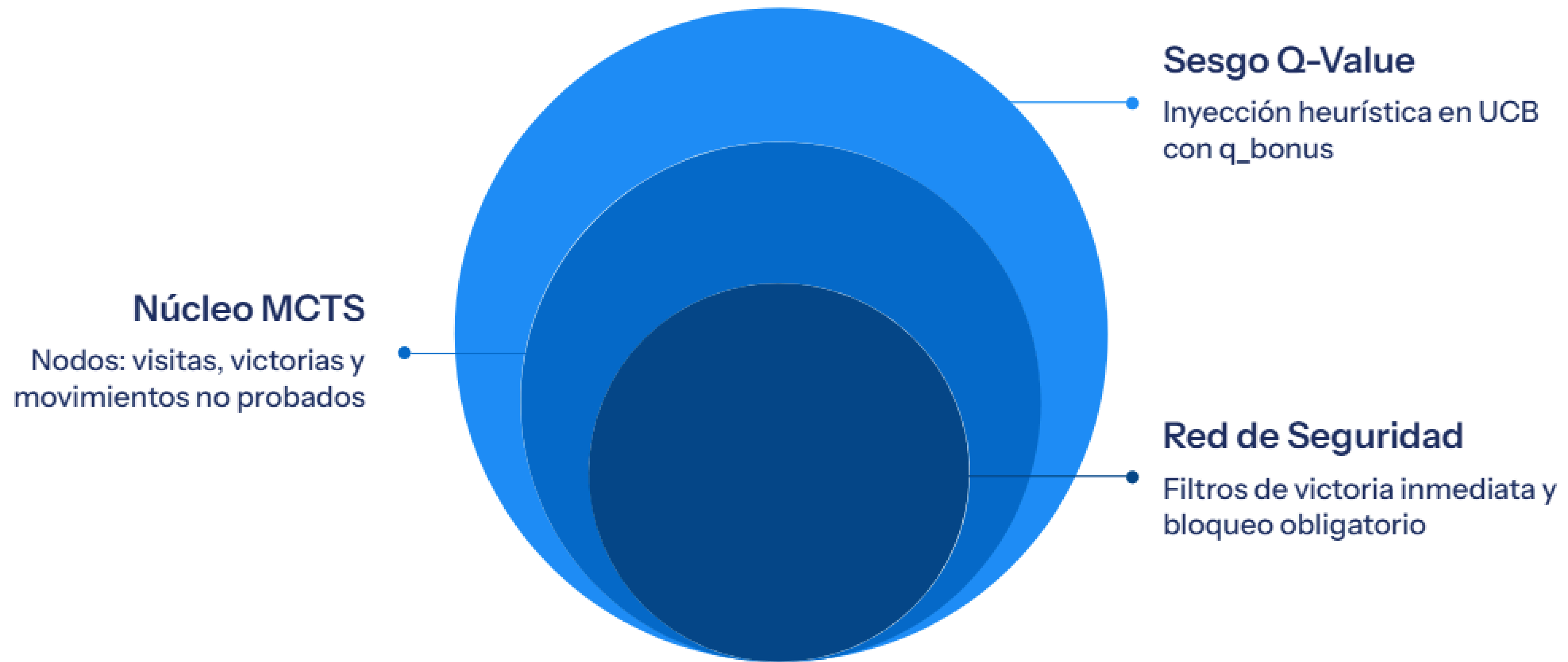
Martin Jerez | Resultados experimentales

Variables: n_episodes de entrenamiento y profundidad Alpha-Beta



Oscar David Vergara | Como funciona el agente

MCTS con Q-bias en la fase de Seleccion (UCB+)



Oscar David Vergara | ¿Por qué es bueno?

MCTS con Q-bias en la fase de Seleccion (UCB+)

Eficiencia

Monitorea el tiempo en caliente con `time.monotonic()` y frena las iteraciones de manera controlada al alcanzar el 85% del tiempo límite ($0.85 * \text{float}(\text{self.action_timeout})$), garantizando que nunca seas descalificado por *timeout*.



Heurísticas Avanzadas

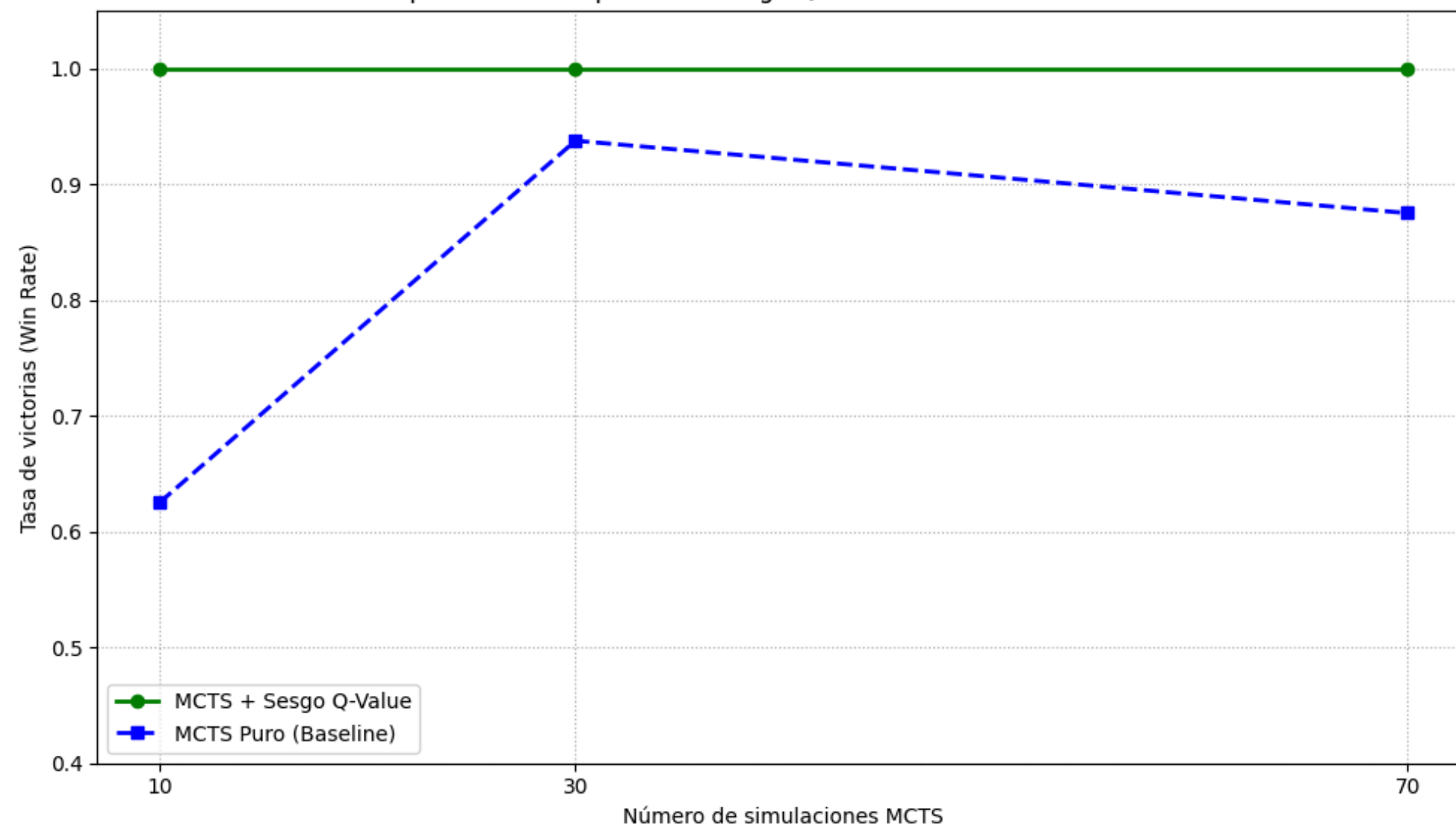
Premia las jugadas usando un sistema de prioridades fijas:

Q_WIN (10.0) y **Q_BLOCK (8.0)** para el control inmediato del fin del juego.
Q_BONUS (2.0) por ocupar la columna central (CENTER_COL), maximizando la conectividad geométrica del tablero.
Control de Amenazas (`_count_open_three_threats`): Escanea las cuatro direcciones (horizontal, vertical y diagonales) buscando estructuras de 3 fichas abiertas. Si la jugada genera un tres en línea propio o bloquea uno del rival, el agente recibe un bono de prioridad.

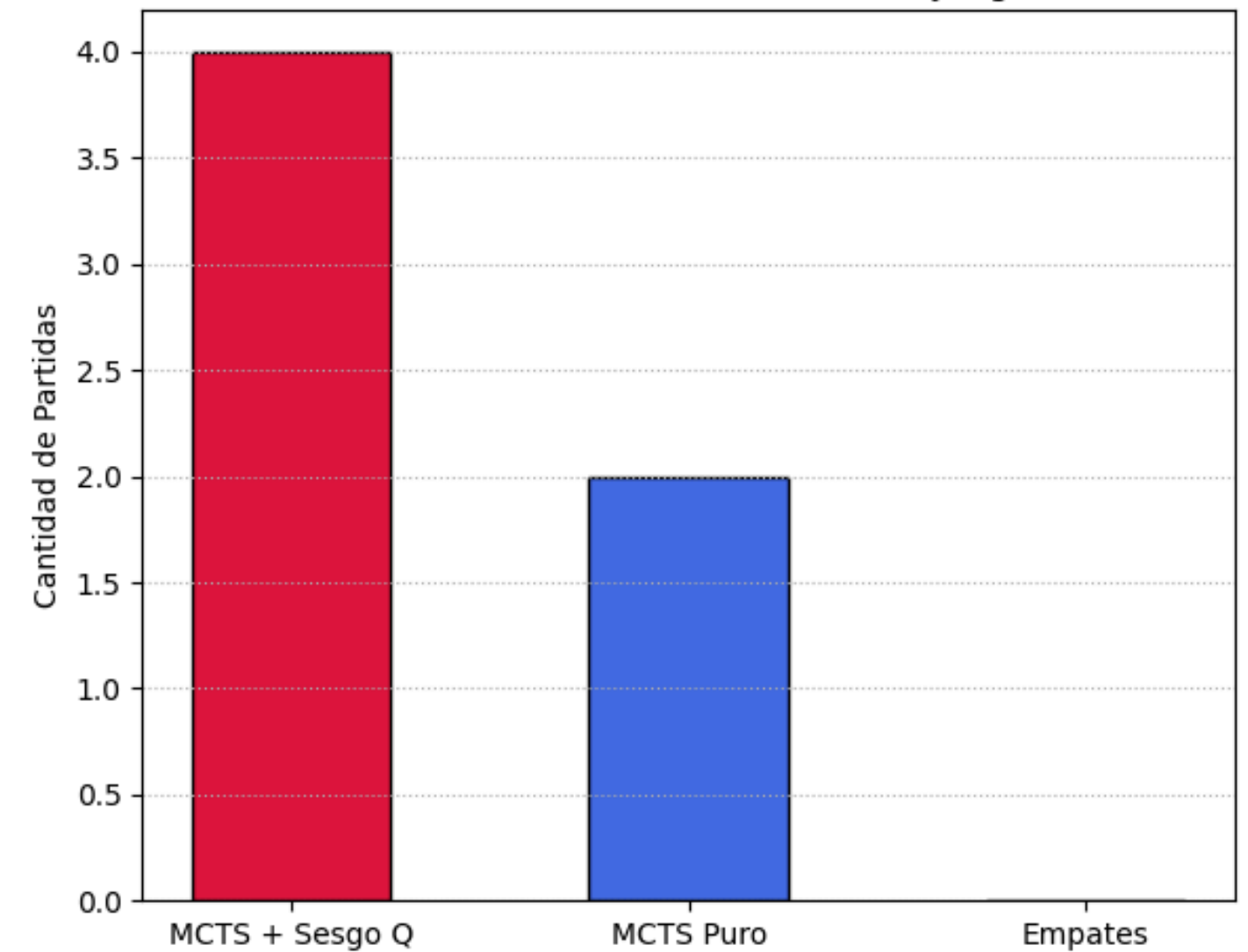
Oscar David Vergara | Resultados experimentales

Variables: num_simulations y use_q_bias (True vs False)

Experimento 1: Aporte del Sesgo Q-Value en la fase de Selección



Distribución de Victorias en Auto-Juego



David Eduardo Lopez | Como funciona el agente

MCTS con rollout heurístico (Aha — MCTSAgentHeuristic)

Arquitectura: MCTS UCT + rollout heurístico

Cuatro fases por cada simulación (ej. 600 o 2000 veces):

- Selección: Recorre el árbol usando la fórmula UCB1 (balance entre explotación y exploración).
- Expansión: Añade un nuevo nodo hijo (movimiento no probado).
- Simulación (Rollout): Juega una partida ficticia hasta el final con una heurística que prioriza:
 - Ganar inmediatamente → si hay movimiento que conecta 4.
 - Bloquear al oponente → si el rival amenaza con ganar.
 - Centro → columnas centrales tienen mayor peso.
- Retropropagación: Actualiza las estadísticas (visitas y victorias) de todos los nodos recorridos.

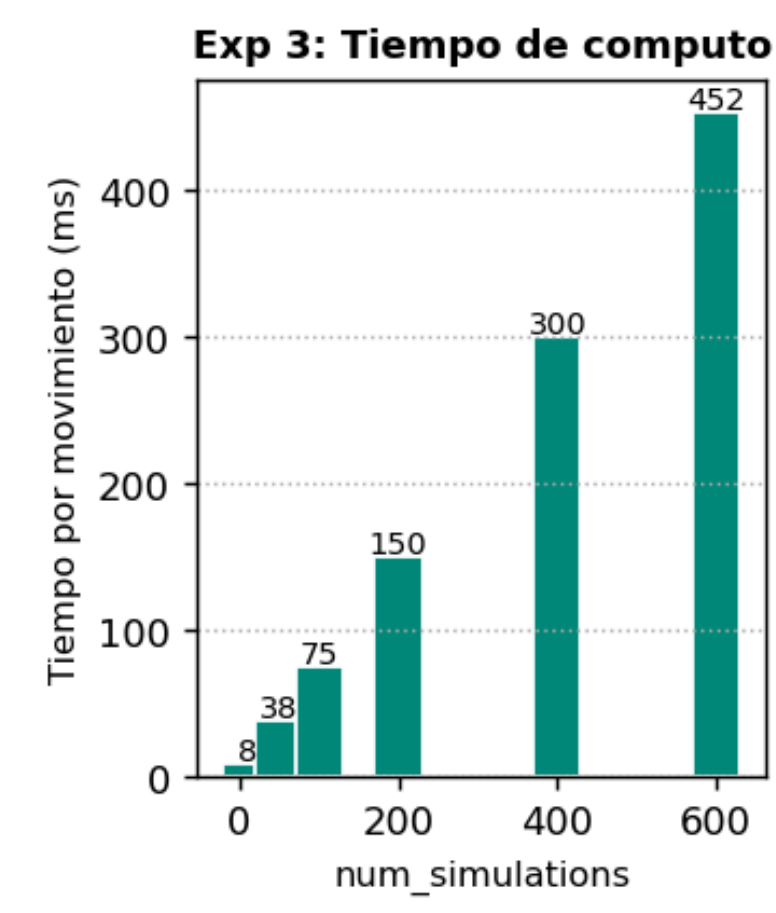
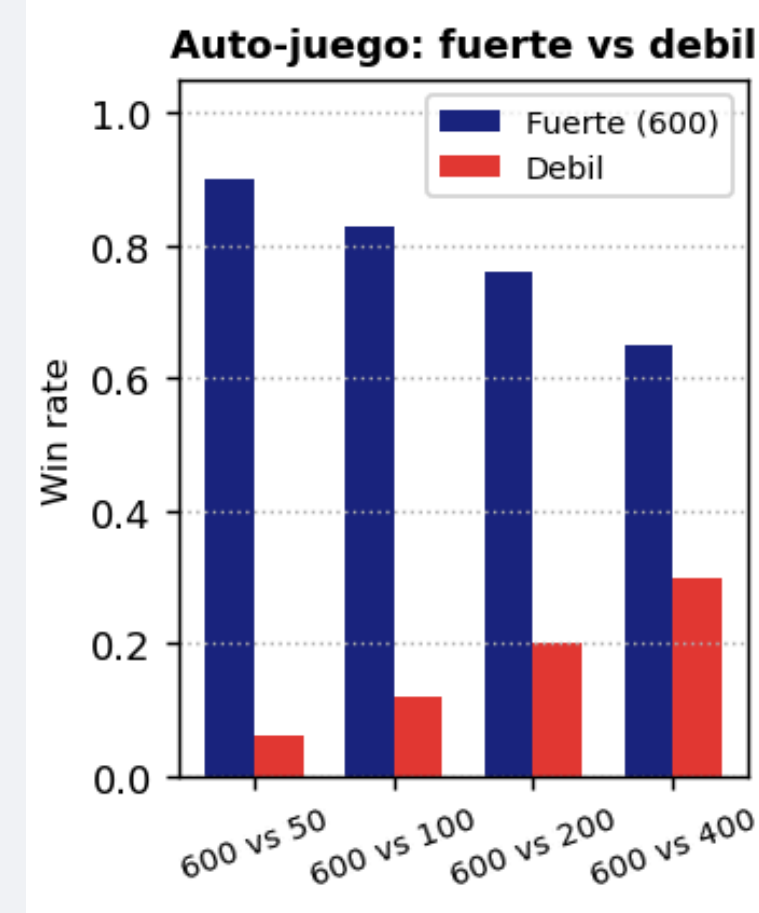
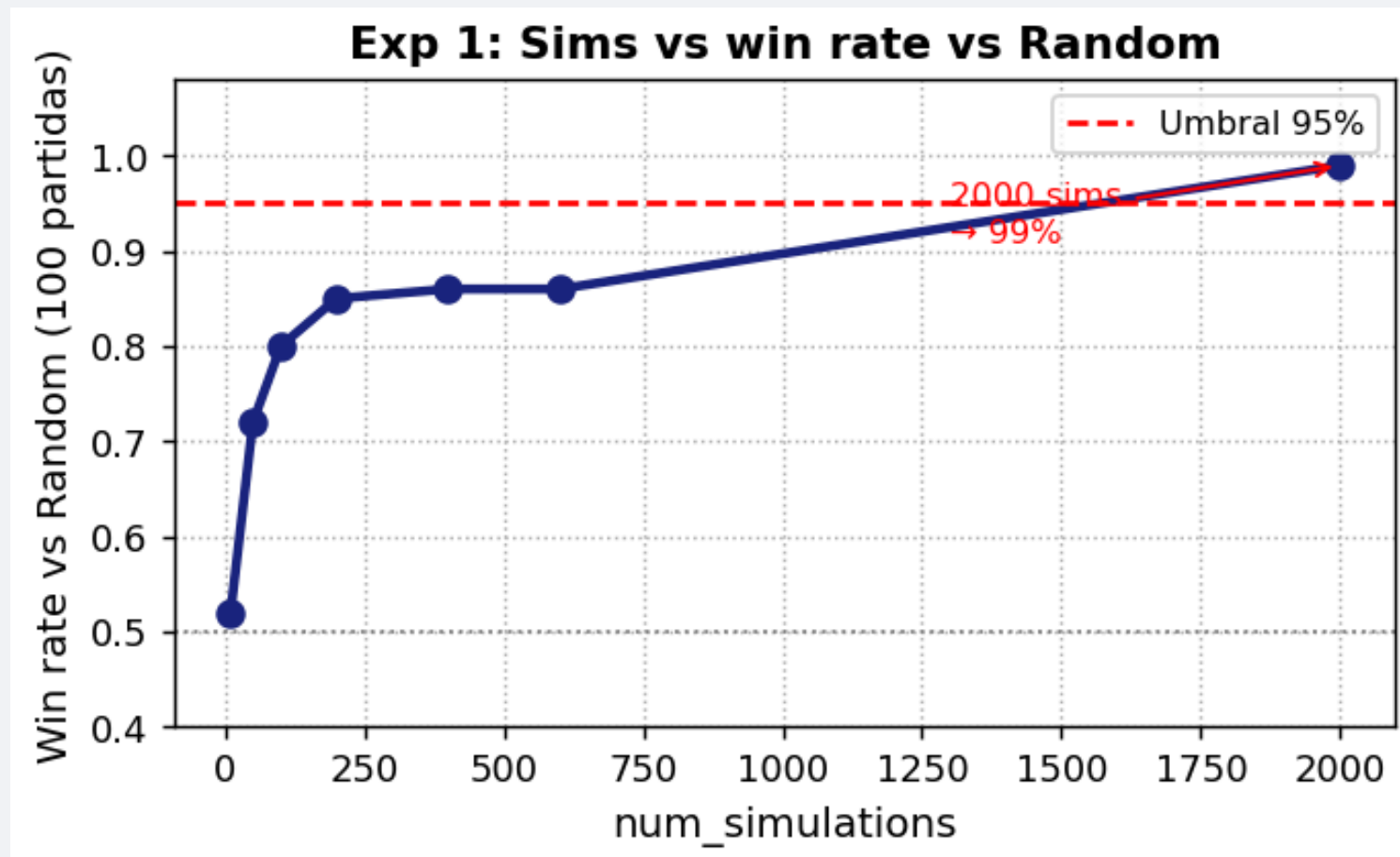
Decisión final: Elige el movimiento con más visitas en la raíz.

Ventajas de Rollout Heurístico

- Nunca ignora una victoria o un bloqueo inmediato
- Favorece el centro determinísticamente
- No requiere partidas de entrenamiento previas
- Consistente — sin varianza de rollout aleatorio

David Eduardo Lopez | Resultados experimentales

Variables: num_simulations y auto-desempeno (fuerte vs debil)



Propuesta de mejora

Cuello de botella identificado:

- El árbol MCTS se reconstruye desde cero en cada movimiento, desperdiciando información de turnos anteriores.

Propuesta concreta:

- Reutilización del árbol: después de ejecutar el mejor movimiento, el nodo hijo correspondiente se convierte en la nueva raíz, conservando todas sus estadísticas (visitas y victorias).

Impacto esperado:

- Reducción del tiempo por movimiento entre un 30% y un 50% para el mismo número de simulaciones.
- Permite aumentar las simulaciones efectivas sin penalización temporal.

¿Por que elegir el agente MCTS + Q-bias?

Argumentos que justifican la seleccion del MCTS + Q-bias para el torneo

1 Rendimiento bajo computo extremo

Con solo 10 simulaciones, Oscar alcanza 100 % vs Random. Martin obtiene ≈60 % y David ≈52 % con el mismo presupuesto computacional. Esto es critico en torneos con limite de tiempo.

2 Superioridad en auto-juego (head-to-head)

Oscar (Q-bias) gana el 66.7 % de las partidas frente a MCTS puro (David) usando el mismo presupuesto de 50 simulaciones. El prior Q-bias guia la busqueda desde la primera simulacion.

3 Cero entrenamiento offline

Oscar no requiere ningun entrenamiento previo: mount() es instantaneo. Martin necesita 5 000 episodios de self-play (~30 s) para obtener pesos utiles. El agente de Oscar es plug-and-play y completamente reproducible.

4 Gestion explicita del timeout del torneo

Oscar usa $0.85 \times \text{action_timeout}$ como presupuesto de tiempo, nunca excede el limite. David puede tardar 452 ms por movimiento con 600 sims y agotar el tiempo. Martin no controla el tiempo de la busqueda Alpha-Beta en profundidades altas.

5 Maxima relacion valor / costo de implementacion

El Q-bias es un cambio minimo a UCB1: $\text{score} = \text{wins}/n + c \cdot \sqrt{\ln N/n} + q/n$. Una sola linea extra produce el mayor salto de rendimiento del grupo.Codigo limpio, sin dependencias adicionales ni hiperparametros extra.